# SecureVault: Encrypted Note & Credential Keeper

Complete Technical Architecture, Security Specifications, Setup & Viva Defense Guide

| | |
|---|---|
| **Project:** SecureVault | **Stack:** Python 3.10+, Flask 3.x, SQLite (No ORM) |
| **Domain:** Full-Stack Cyber Defense | **Crypto:** AES-256-GCM + HKDF-SHA256 + scrypt |

## 1. Executive Summary & Purpose

**SecureVault** is an academic-grade, secure web application designed to store confidential notes and sensitive login credentials. Unlike commercial applications that obscure cryptographic primitives behind heavy abstractions, SecureVault was purposefully built to **demonstrate transparent, auditable, defense-in-depth cybersecurity controls**.

All sensitive data stored in the database is encrypted at rest using individual authenticated ciphertexts. Passwords are never stored in plaintext, and every layer of the HTTP request lifecycle enforces rigorous defensive controls against injection, cross-site request forgery, cross-site scripting, session hijacking, and automated brute-force attacks.

## 2. System Architecture & Workflow

SecureVault is structured following the Flask Application Factory pattern, using raw SQLite queries with strict parameterized inputs. The cryptographic workflow follows a zero-plaintext at rest model:

| Phase | Component | Security Operation & Data Flow |
|---|---|---|
| Registration | app/auth.py | Enforces password complexity (10-128 chars, upper, lower, digit, symbol). Generates 16 random salt by |
| Key Derivation | app/crypto.py | HKDF-SHA256 extracts a unique 256-bit AES user key using the 32-byte master key (.env) + that user's |
| Data Storage | app/vault.py | AES-256-GCM encrypts each field with a fresh 12-byte random nonce and associated data 'user:<id>'. |
| Data Retrieval | app/vault.py | Queries fetch raw rows by 'user_id = ?'. AES-256-GCM decrypts payload in server memory. Passwords |
| Password Reveal | app/vault.py | Dedicated reveal endpoint decrypts on demand with rate limiting (30/min). Frontend reveals plaintext for |

# 3. The Five Core Security Mechanisms (In Depth)

## Mechanism 1: AES-256-GCM Encryption at Rest (app/crypto.py)

• **Master Key:** A 32-byte cryptographically secure random key stored in .env (never in SQLite, never committed to git).
• **Per-User Key Isolation:** Derives a dedicated 256-bit AES key per user using HKDF-SHA256 with user-specific 16-byte random salt and context info=b'securevault-user-key-v1'.
• **Authenticated Cipher:** Uses AES in Galois/Counter Mode (GCM). Generates a fresh 12-byte random nonce for *every single field*.
• **Cryptographic Binding:** Associated Authenticated Data (AAD) binds the ciphertext to user:{user_id}, preventing cross-tenant ciphertext swapping.
• **Tamper Detection:** Any alteration of ciphertext bytes fails GCM tag authentication, throwing an exception and logging an audit event.

> **THREAT MITIGATED:** Threat Mitigated: Database Exfiltration & Storage Tampering. If an attacker dumps the SQLite file, they obtain only high-entropy random-looking ciphertext blobs.

## Mechanism 2: Salted Password Hashing & Timing Mitigation (app/auth.py)

• **scrypt Hashing:** Uses werkzeug's scrypt implementation (memory-hard, resistant to GPU/ASIC cracking) with fallback to PBKDF2:SHA256 (600,000 iterations).
• **Strict Password Policy:** Minimum 10 characters, at least 1 uppercase, 1 lowercase, 1 digit, 1 special symbol, max 128 characters.
• **Constant-Time Dummy Verification:** When a non-existent username attempts login, the server still verifies against a pre-computed dummy hash. Returns identical generic 'Invalid username or password' responses to prevent timing enumeration.
• **Account Lockout:** 5 consecutive failed login attempts trigger an automatic 5-minute lockout.

> **THREAT MITIGATED:** Threat Mitigated: User Enumeration & Offline Hash Cracking. Eliminates response-timing discrepancy between valid and invalid usernames.

## Mechanism 3: Tiered IP-Based Rate Limiting (app/security.py)

• **Flask-Limiter Keying:** Rate limits are strictly tied to remote client IP addresses (get_remote_address).
• **Granular Throttling:** POST /login: 5/minute; POST /register: 5/hour; /reveal: 30/minute; default: 200/hour.
• **Graceful 429 Responses:** Returns sanitized HTTP 429 JSON/HTML and logs an immutable rate_limited audit event.

> **THREAT MITIGATED:** Threat Mitigated: Automated Credential Stuffing & Denial of Service (DoS). Throttles password-guessing bots to an infeasible rate.

## Mechanism 4: SQL Injection & Cross-Site Scripting (XSS) Defenses

• **100% Parameterized SQL:** Raw SQLite queries strictly use ? placeholders. String concatenation and f-strings in SQL are completely forbidden.
• **Strict Content Security Policy (CSP):** Outbound headers enforce default-src 'self'; script-src 'self'; style-src 'self'; object-src 'none'; frame-ancestors 'none'.
• **Safe DOM Manipulation:** Frontend uses textContent and createElement exclusively. innerHTML, outerHTML, eval(), and inline handlers (onclick) are strictly absent.
• **Defensive Headers:** X-Content-Type-Options: nosniff, X-Frame-Options: DENY, Referrer-Policy: no-referrer.

> **THREAT MITIGATED:** Threat Mitigated: SQL Injection, Stored/Reflected XSS, and Clickjacking. Payloads like ' OR 1=1-- or alert(1) are treated as inert strings.

## Mechanism 5: Hardened Session Management & CSRF Defense

• **Cookie Hardening:** Signed Flask cookies with HttpOnly=True, SameSite='Lax', and configurable COOKIE_SECURE.
• **Idle Timeout (15 Mins):** Sliding inactivity window tracked server-side and client-side with countdown and 60-second warning banner.
• **Session Fixation Defense:** session.clear() is executed on successful login before assigning fresh session keys.
• **Per-Session Anti-CSRF Token:** Every state-changing request (POST/PUT/DELETE) must send a valid CSRF token, compared via constant-time hmac.compare_digest.

> **THREAT MITIGATED:** Threat Mitigated: Cross-Site Request Forgery (CSRF), Session Fixation, and Session Hijacking via Malicious Scripts.

## Bonus Defensive Controls

• **Insecure Direct Object Reference (IDOR) Protection:** Every record lookup includes AND user_id = ?. Accessing another user's note or credential returns **404 Not Found** (never 403), preventing attackers from enumerating valid resource IDs.
• **Append-Only Audit Log:** Tracks register, login_success, login_failed, account_locked, credential_revealed, etc., with client IP and UTC timestamp without leaking secrets.
• **Sanitized Error Pages:** HTTP errors (400, 401, 403, 404, 405, 413, 429, 500) return sanitized error views; Werkzeug debuggers and stack traces are suppressed.

# 4. How to Make It Work (Setup, Run & Verify)

The project is structured to run out of the box with zero external internet dependencies during runtime (no CDN scripts, no external fonts).

# 5. How to Explain It in a Viva / Evaluation

Use this structured framework when presenting SecureVault to an evaluator, professor, or interviewer.

## The 2-Minute Elevator Pitch

*"SecureVault is an encrypted note and credential keeper designed to demonstrate end-to-end defense-in-depth principles. Unlike standard web applications where database administrators or attackers with SQL dump access can read plaintext secrets, SecureVault enforces zero plaintext at rest using authenticated AES-256-GCM. We derive per-user keys using HKDF-SHA256, hash passwords using memory-hard scrypt, mitigate timing attacks using constant-time dummy verifications, protect against brute-force using tiered rate limiting and account lockouts, eliminate injection via 100% parameterized queries, and protect sessions with strict sliding idle timeouts and CSRF tokens. The application has 15 automated security tests verifying each defense mechanism."*

## 5-Minute Live Demonstration Checklist

| Step | Action | What to Point Out to the Professor |
|---|---|---|
| 1 | Go to /register and type a weak password | Show the live complexity checklist dynamically rejecting passwords lacking uppercase/numbers/symbo |
| 2 | Submit compliant registration & log in | Explain that passwords are saved as scrypt salted hashes, and a unique 16-byte HKDF salt is created. |
| 3 | Add a secret note & credential | Show that passwords are saved encrypted and masked in the UI with dots (••••••••). |
| 4 | Click 'Reveal' on credential | Show the 10-second countdown timer before the password automatically re-masks itself. |
| 5 | Click 'View Ciphertext' / 'Raw' | Show the raw base64 string. Explain the payload format: nonce + ciphertext + GCM tag. |
| 6 | Run 'python scripts/show_db.py' | Show the terminal output proving SQLite has zero plaintext secrets and passwords are only scrypt hash |
| 7 | Try SQL injection on login: ' OR '1'='1 | Show that the attempt returns 401 without error 500, proving parameterized queries work. |
| 8 | Try XSS payload: <script>alert(1)</script> | Show that it renders safely as plain text because of textContent and strict CSP headers. |
| 9 | Trigger Rate Limiting (6 rapid logins) | Show the application returning HTTP 429 Too Many Requests and logging an audit event. |

## Top 8 Anticipated Viva Questions & Model Answers

### Q1: Why did you choose AES-GCM over AES-CBC?

**A:** AES-CBC only provides confidentiality, making it vulnerable to padding oracle attacks and bit-flipping if unauthenticated. AES-GCM is an Authenticated Encryption with Associated Data (AEAD) mode that provides both confidentiality and cryptographic integrity. If any ciphertext byte is tampered with, GCM tag verification fails immediately.

### Q2: What is HKDF and why derive per-user keys instead of using the master key directly?

**A:** HKDF (HMAC-based Key Derivation Function, RFC 5869) extracts and expands entropy. Deriving per-user keys ensures cryptographic domain separation. Even if one user's derived key were ever compromised, other users' data remains completely secure.

### Q3: How do you prevent timing attacks during user login?

**A:** When an unregistered username is entered, we still run check_password_hash() against a pre-computed dummy hash. This ensures the server takes the exact same mathematical computation time regardless of whether a username exists, defeating user enumeration.

### Q4: Why return 404 instead of 403 when User B tries to access User A's note (IDOR)?

**A:** Returning 403 confirms to an attacker that the resource exists (ID enumeration). Returning 404 treats unauthorized records as non-existent, revealing zero intelligence about valid ID ranges.

### Q5: Why did you not use an ORM like SQLAlchemy?

**A:** ORMs can hide query mechanics and introduce complex dependencies. In cybersecurity evaluations, writing parameterized SQL statements by hand with explicit ? placeholders proves an exact understanding of SQL injection prevention at the transport layer.

### Q6: How does your Content Security Policy (CSP) prevent XSS?

**A:** Our CSP specifies default-src 'self'; script-src 'self'; object-src 'none'. Browsers strictly refuse to execute inline scripts (<script> tags or onclick attributes) or scripts loaded from untrusted domains, neutralizing injected scripts.

### Q7: How is session fixation mitigated?

**A:** Upon successful login, the application explicitly calls session.clear() before establishing fresh user authentication keys. This destroys any pre-existing session token an attacker might have attempted to fixate.

### Q8: What are the main real-world limitations and future improvements?

**A:** In a production enterprise system, the master key would reside in a Hardware Security Module (HSM) or Cloud KMS (AWS KMS / HashiCorp Vault) rather than a local .env file. Additional improvements include TOTP two-factor authentication, Redis-backed rate limiting, and automated key rotation.